

20/8/22

OS LAB

1

Basic Linux Commands

- 1] man - help
- 2] pwd - present working directory
- 3] cd
- 4] mkdir - create dir
- 5] rmdir - remove dir
- 6] ls
- 7] ls -l - long listing
- 8] touch
- 9] cat > file1.txt - create
- 10] ls
- 11] cat file1.txt - view f tac
cat filename - display in reverse
- 12] cat >> file1.txt
- 13] mv - move file dir
- 14] cp - cp file1 file2
- 15] rm - rm file1
- 16] wc
- 17] date - find date
- 18] cal
- 19] head - head - 5 date 1 = display 1st 5 lines
- 20] tail - tail - 5 = display 1st 5 lines from end
- 21] cat file1.txt | wc l - pipe
- 22] cat file1.txt | tee a.txt
- 23] grep - grep - c 'the' file name c - word
n - line nos
- 24] paste - join 2 files - i
- 25] cut - cut - c 1,3 file exclude line nos
- 26] chmod 0 - no permission 1 = execute 2 = write 4 = read
 - chmod u+x
 - chmod a+x
 - chmod a-x

chmod 424 file.txt
chmod 777 file.txt
ugo

5/9/22

Shell scripting

2

PAGE No.

DATE

variables are 'untyped' in shell script i.e. we don't define datatype

extension for shell scripting file = **sh**

nano sl.sh - creates sl.sh script and opens nano editor to write commands in script

1] `ls -l sl.sh` → notice that file has only read-write permission
`chmod u+x sl.sh` → give 'execute' permission.
`ls -l sl.sh`
`./sl.sh`

* Note Don't give spaces bet '='
 i.e. `val1=20`

2] `bash sl.sh`

`val1=20`

`echo "$val1"` → \$ sign is used to display contents stored in variable

x - execute permission
 r - read permission
 w - write permission

no space

<code>chmod</code>	<code>u</code>	<code>+</code>	<code>x</code>
(owner,	<code>g</code>	<code>-</code>	<code>w</code>
group,	<code>o</code>	<code>=</code>	<code>x</code>
other,	<code>a</code>		
all)		<code>+</code> = add mode	
		<code>-</code> = remove mode	
		<code>=</code> = exact mode	

`val1=20`

`echo "$val1"`

`read file1`

`// mkdir { $1, $2 }` → passed as argument in bash

`echo "$@"`

→ Print all

`bash prog.sh abc xyz`

alias `a1=""`

`a1 capitals`

touch \$1 \$2.

echo "\$1" - 1st positional parameter

echo "\$#" " \$*"
 ↓ ↓
 No of All pos parameters
 positional parameters

→ bash sh 11 12

if [\$num1 -gt \$num2]
 then

echo "\$num1 is greater"

else

echo "\$num2 is greater"

end of
 for -
 loop fi

for ((i=1; i < \$size; i++))

do

:

done

12/9/22

OS Lab 3

Arithmetic operation

3

Q) write a script to perform different arithmetic operations using simple calculator.

Arithmetic operation

1

num1 = 6

+, -, *, /, %, num2 = 3 *no \$ sign

**

sum = \$(num1 + num2)

put \$ here

echo "sum = \$sum"

simplecalculator1.sh

2 num1 = 6

num2 = 3

sum = \$(^{single bracket}expr num1 + num2)

echo "sum = \$sum"

simplecalculator2.sh

For multiply

/*

**

not available

Arrays

→ a = (100 200 300 400 500)

echo \${a[@]} → print all array elements

echo \${a[1]} → print second element

no space

Accept array
→

Store in array using loop

for ((i=0; i<\${size}; i++))

do

read b

len = \${#c}

c[\$i]=\$b

done

Reverse string

for ((i=\${#str}-1; i>=0; i--))

do

temp=\${str:i:1}

str2=\$str2\$temp

done

```
#bin/bash
```

```
echo "Enter 1st number"
```

```
read num1
```

```
echo "Enter 2nd number"
```

```
read num2
```

```
sum=$((num1+num2))
```

```
difference=$((num1-num2))
```

```
product=$((num1*num2))
```

```
Quotient=$((num1/num2))
```

```
exponent=$((num1**num2))
```

```
remainder=$((num1%num2))
```

```
echo "Sum= $sum"
```

```
echo "Difference= $difference"
```

```
echo "product= $product"
```

```
echo "Quotient= $Quotient"
```

```
echo "Exponent= $exponent"
```

```
echo "Remainder= $remainder"
```

```
#bin/bash
```

```
echo "Enter First Number"
```

```
read num1
```

```
echo "Enter Second Number"
```

```
read num2
```

```
sum=$(expr $num1 + $num2)
```

```
difference=$(expr $num1 - $num2)
```

```
product=$(expr $num1 \* $num2)
```

```
Quotient=$(expr $num1 / $num2)
```

```
Remainder=$(expr $num1 % $num2)
```

```
echo "Sum = $sum"
```

```
echo "Difference = $difference"
```

```
echo "product = $product"
```

```
echo "Quotient = $Quotient"
```

```
echo "Remainder = $Remainder"
```